



Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3067 Redes

Proyecto 1: Uso de un protocolo existente

Model Context Protocol

Entrega parcial. Incisos 4 y 5 de la primera parte, e incisos 8 y 10 del reporte aplicados a esa misma parte.

Repositorio: github.com/UVG-Work/Proyecto-1-Chat-MCP-

El proyecto consiste en un chatbot que funciona como anfitrión de MCP y utiliza las herramientas expuestas por varios servidores. El protocolo se implementó a mano: el formato e intercambio de mensajes JSON-RPC 2.0, la fase de inicialización, la negociación de versión y el transporte stdio se escribieron desde cero, tanto del lado del cliente como del servidor, sin utilizar ningún SDK de MCP. Este documento cubre únicamente los incisos indicados en la carátula.

Inciso 4. Uso de servidores MCP locales existentes

El anfitrión se conecta a los dos servidores oficiales mediante el transporte stdio: el cliente lanza cada servidor como un subproceso e intercambia mensajes JSON-RPC delimitados por saltos de línea a través de su entrada y salida estándar.

Servidor	Implementación	Invocación	Herramientas
Filesystem	secure-filesystem-server 0.2.0	<code>npx -y @modelcontextprotocol/server-filesystem ./sandbox</code>	14
Git	mcp-git 1.29.0	<code>uvx mcp-server-git</code>	12

Ambos negocian la revisión 2025-11-25 del protocolo y declaran la capacidad `tools`. El servidor de Git es un paquete de Python, por lo que el proyecto requiere `uv` instalado aunque el resto esté escrito en TypeScript.

Escenario demostrado

Se solicita al chatbot, en lenguaje natural, que cree un archivo README, lo agregue al repositorio y realice un commit. El modelo decide por sí mismo qué herramientas usar y en qué orden; el anfitrión únicamente traduce esas decisiones en llamadas `tools/call` y las enruta al servidor que corresponde.

Instrucción: "Inside the demo-repo directory, create a file named README describing this project. Then add it to the git repository in that same directory and commit it with a descriptive message. Finally show me the

```
filesystem__list_allowed_directories    {}                                ok
filesystem__list_directory              {"path":".../sandbox"}           ok
filesystem__write_file                  {"path":".../README.md"}         ok
git__git_status                        {"repo_path":"..."}              ok
git__git_add                           {"files":["README.md"]}           ok
git__git_commit                        {"message":"Add README..."}       ok
git__git_log                           {"repo_path":"..."}              ok
```

Resultado verificado en disco:

```
dc5c5e7  Add README.md describing the MCP chat host for networking co
README.md | 22 ++++++
```

El commit existe realmente en el repositorio de prueba y fue creado por el chatbot a través de MCP, no desde la línea de comandos. El modelo combina además dos servidores distintos en una sola tarea: escribe el archivo con el servidor de Filesystem y lo versiona con el de Git.

Limitación encontrada

El servidor oficial de Git, en su versión 1.29.0, no expone una herramienta `git_init`. Las doce herramientas disponibles son `git_status`, `git_diff`, `git_diff_staged`, `git_diff_unstaged`, `git_commit`, `git_add`, `git_reset`, `git_log`, `git_create_branch`, `git_checkout`, `git_show` y `git_branch`, ninguna de las cuales crea un repositorio. Por esa razón la creación del repositorio se realiza mediante un script de preparación, mientras que la creación del README, su incorporación al índice y el commit sí se ejecutan por medio de MCP.

Inciso 5. Servidor MCP propio

Caso de uso a nivel de industria

El servidor implementa la mesa de servicio de primer nivel de un proveedor de internet. Cuando un abonado llama reportando lentitud, el agente debe identificar la cuenta, consultar la telemetría del circuito, descartar una avería de zona conocida y decidir si la evidencia justifica enviar a un técnico. En un operador real cada una de esas consultas corresponde a un sistema distinto.

Ese encadenamiento es lo que hace al caso apropiado para MCP: el modelo no invoca una sola herramienta, sino que ordena cinco y decide en cada paso qué hacer con el resultado anterior. Todos los datos son sintéticos y los diagnósticos se basan en umbrales determinísticos, de modo que una demostración produce siempre el mismo resultado.

Herramientas expuestas

Herramienta	Propósito
<code>lookup_subscriber</code>	Localiza una cuenta por nombre, identificador, circuito o teléfono. Devuelve el identificador de circuito que requieren las demás herramientas.
<code>get_link_metrics</code>	Telemetría del enlace: latencia, jitter, pérdida de paquetes, relación señal a ruido, throughput medido, tiempo en servicio y número de caídas.
<code>run_link_diagnostics</code>	Evalúa la telemetría contra los umbrales del operador y devuelve un diagnóstico con la acción recomendada.
<code>check_zone_outage</code>	Incidentes conocidos que afectan una zona de servicio, con causa, estado y tiempo estimado de resolución.
<code>open_incident_ticket</code>	Escala el caso creando un ticket de incidente. Es la única herramienta que modifica estado.

Escenario demostrado

Instrucción: "La clienta Maria Elena Ramirez llama porque su internet es lento. Investiga que ocurre con su servicio y, si corresponde, abre un ticket."

lookup_subscriber

{"query":"Maria Elena Ramirez"}

ok

get_link_metrics

{"circuit_id":"GT-CIR-004821"}

ok

check_zone_outage

{"zone":"ZONA-MIXCO-03"}

ok

run_link_diagnostics

{"circuit_id":"GT-CIR-004821"}

ok

open_incident_ticket

{"subscriber_id":"SUB-100482", ...}

ok

Hallazgos: pérdida de paquetes 8.2 % (umbral crítico 5 %), SNR 14.1 dB (umbral crítico 15 dB), jitter 19.4 ms, 11 caídas en 24 h, throughput 62 de 300 kbps.

Sin incidencias de zona que expliquen el síntoma.

Diagnóstico: falla crítica del enlace. Ticket TCK-4401 abierto, severidad asignado a Field Operations.

El modelo secuencia las cinco llamadas por su cuenta a partir de la descripción del problema y de las descripciones de las herramientas. No existe en el anfitrión ninguna lógica que codifique ese orden.

Datos de prueba

El conjunto de datos está construido de modo que cada abonado ejerce una ruta diagnóstica distinta, lo que permite demostrar los cuatro diagnósticos posibles.

Abonado	Circuito	Condición
SUB-100482	GT-CIR-004821	Degradado: 8.2 % de pérdida, SNR 14.1 dB, 11 caídas
SUB-100731	GT-CIR-005190	Sano: todas las métricas dentro de umbral
SUB-100955	GT-CIR-005544	Caído por avería de zona activa
SUB-101204	GT-CIR-006012	Enlace sano pero cuenta suspendida
SUB-101488	GT-CIR-006377	Congestión: 137 ms de latencia y 42.6 ms de jitter, sin pérdida

Los dos últimos casos son deliberados. Una cuenta suspendida y un corte de fibra se parecen, vistos únicamente desde la telemetría, a un enlace averiado; enviar un técnico en cualquiera de los dos casos sería un desplazamiento inútil, por lo que el diagnóstico los distingue de forma explícita.

Inciso 8. Especificación del servidor desarrollado

Nombre del servidor	noc-support-desk
Versión	1.0.0
Revisión de MCP	2025-11-25, con negociación hacia atrás a 2025-06-18, 2025-03-26 y 2024-11-05
Capacidades declaradas	tools, con listChanged: false
Transporte	stdio
Punto de entrada	src/server/stdio-main.ts

Métodos soportados

Método	Tipo	Comportamiento
initialize	solicitud	Devuelve la versión negociada, las capacidades, la identificación del servidor y sus instrucciones de uso.
notifications/initialized	notificación	Se consume y nunca se responde, conforme a JSON-RPC.
ping	solicitud	Devuelve un resultado vacío.
tools/list	solicitud	Devuelve las cinco herramientas con su esquema de entrada.
tools/call	solicitud	Ejecuta una herramienta y devuelve su resultado.

Cualquier otro método devuelve el error JSON-RPC -32601, método no encontrado.

Parámetros de las herramientas

Herramienta	Parámetro	Tipo	Descripción
lookup_subscriber	query	string	Nombre, identificador SUB-#####, circuito GT-CIR-##### o teléfono. Requerido.
get_link_metrics	circuit_id	string	Identificador de circuito. Requerido.
run_link_diagnostics	circuit_id	string	Identificador de circuito. Requerido.
check_zone_outage	zone	string	Zona de servicio, por ejemplo ZONA-GUATE-07. Requerido.
open_incident_ticket	subscriber_id	string	Identificador exacto del abonado. Requerido.
	summary	string	Descripción del fallo y su evidencia. Requerido.
	severity	string	Uno de low, medium, high o critical. Requerido.

Umbrales de diagnóstico

Métrica	Degradado	Crítico
Pérdida de paquetes	mayor o igual a 1 %	mayor o igual a 5 %
Latencia	mayor o igual a 60 ms	mayor o igual a 120 ms
Jitter	mayor o igual a 15 ms	mayor o igual a 30 ms
Relación señal a ruido	menor o igual a 20 dB	menor o igual a 15 dB
Caídas en 24 horas	mayor o igual a 5	no aplica

Los diagnósticos posibles son los siguientes.

Diagnóstico	Significado y acción recomendada
healthy	Todo dentro de umbral. Revisar el equipo del cliente antes de escalar.
link-fault	Uno o más umbrales excedidos. Abrir ticket con la severidad correspondiente.
zone-outage	Una avería conocida explica el síntoma. Informar el tiempo estimado y no duplicar el ticket.
account-suspended	El enlace está sano pero la cuenta está suspendida. Derivar a facturación.

Endpoints y transporte

En esta primera parte el servidor se ejecuta localmente sobre stdio, por lo que no expone endpoints de red. Las reglas de trama son las que fija la especificación:

- un mensaje JSON-RPC por línea, codificado en UTF-8;
- los mensajes no pueden contener saltos de línea incrustados;
- la salida estándar transporta únicamente mensajes MCP, por lo que todo diagnóstico del servidor se escribe en la salida de error;
- el cliente no debe interpretar la salida de error como una condición de fallo;
- cerrar la entrada estándar termina el servidor de forma ordenada.

Manejo de errores

Código	Significado	Cuándo se produce
-32700	Error de análisis	La carga no es JSON válido.
-32600	Solicitud inválida	No es un mensaje JSON-RPC 2.0 válido.
-32601	Método no encontrado	Método desconocido.
-32602	Parámetros inválidos	Herramienta inexistente, argumento requerido ausente, tipo incorrecto o valor fuera del conjunto permitido.
-32603	Error interno	Fallo inesperado del servidor.

Conviene distinguir dos situaciones que suelen confundirse. Una herramienta que se ejecuta y falla no produce un error de JSON-RPC: devuelve una respuesta exitosa cuyo resultado lleva `isError: true` junto con un texto explicativo, de modo que el modelo pueda leer el fallo y reaccionar. Los códigos anteriores se reservan para errores de protocolo.

Ejemplo de intercambio

```
--> {"jsonrpc":"2.0","id":1,"method":"initialize",
    "params":{"protocolVersion":"2025-11-25","capabilities":{},
        "clientInfo":{"name":"uvg-mcp-chat-host","version":"1.0.0"}
    }}

<-- {"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2025-11-25",
    "capabilities":{"tools":{"listChanged":false}},
    "serverInfo":{"name":"noc-support-desk","version":"1.0.0"},
    "instructions":"This server exposes the first-line support desk..."}

--> {"jsonrpc":"2.0","method":"notifications/initialized"}

--> {"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}
<-- {"jsonrpc":"2.0","id":2,"result":{"tools":[ ... 5 herramientas ... ]}}

--> {"jsonrpc":"2.0","id":3,"method":"tools/call",
    "params":{"name":"lookup_subscriber",
        "arguments":{"query":"Maria Elena Ramirez"}}}
<-- {"jsonrpc":"2.0","id":3,"result":{"content":[{"type":"text",
    "text":"Found 1 matching subscriber(s): SUB-100482 ..."}],
    "structuredContent":{"matches":[ ... ]}}}
```

El campo `instructions` merece una nota. El servidor describe en él el orden recomendado de uso de sus herramientas y el anfitrión lo reenvía al modelo, de modo que el servidor documenta su propio uso a través del protocolo en lugar de que el anfitrión codifique conocimiento sobre un servidor en particular.

Inciso 10. Conclusiones y comentarios

MCP es una convención y no un formato de trama. No existe una cabecera de MCP ni un puerto de MCP: lo que hay es JSON-RPC 2.0 con un conjunto acordado de nombres de método, un ciclo de vida y una negociación de capacidades. Esa es precisamente la razón por la que resulta viable implementarlo a mano y por la que el mismo código de servidor puede atender transportes distintos sin modificarse.

La interoperabilidad es el objetivo del protocolo y solo se comprueba al conectarse con servidores ajenos. Escribir un servidor propio es cómodo porque uno responde a los mensajes que uno mismo eligió; el inciso 4 obliga a lo contrario, y es ahí donde cada ambigüedad de la especificación se convierte en un fallo concreto.

La negociación de versión no puede fijarse. La intuición inicial es enviar la revisión más reciente y esperar la misma de vuelta, pero la especificación establece que el servidor responde con su revisión preferida y que el cliente debe aceptarla si la soporta o desconectarse en caso contrario. Un cliente con una única versión fija funciona contra el servidor propio y falla contra los oficiales, que es el peor tipo de error porque las pruebas locales nunca lo revelan.

La salida estándar es sagrada en stdio. Un servidor no debe escribir en ella nada que no sea un mensaje MCP, pues un simple mensaje de depuración corrompe la trama y rompe la sesión completa. Por eso todos los diagnósticos del servidor propio se escriben en la salida de error, y el cliente trata esa salida como informativa y nunca como una señal de fallo.

Las tramas deben reensamblarse y no suponerse. Un solo evento de lectura sobre el flujo puede contener medio mensaje, varios mensajes o ambas cosas, de manera que el transporte acumula lo recibido, divide en los saltos de línea y conserva el resto para la lectura siguiente.

Lanzar los servidores oficiales en Windows tiene su detalle. Tanto `npx` como `uvx` son archivos con extensión `.cmd`, que Node no puede ejecutar directamente por una restricción de seguridad introducida a raíz de CVE-2024-27980, por lo que es necesario pasar por el intérprete de comandos y encargarse del entrecomillado de los argumentos de forma explícita.

Diseñar el transporte como una interfaz paga desde el primer día. Separarlo del cliente y del núcleo del servidor permite que el mismo objeto de servidor atienda stdio y HTTP sin duplicar lógica; si esa separación se hubiera pospuesto, la segunda parte del proyecto habría implicado reescribir el servidor en lugar de añadirle un punto de entrada.

Un solo punto de registro produce un registro completo. Al capturar cada trama en la capa de transporte, el requisito de mantener y mostrar un log de todas las interacciones se satisface por

construcción y no depende de recordar registrar en cada lugar del código.

Un buen caso de uso para un agente es aquel que exige encadenar decisiones. Una herramienta aislada no demuestra gran cosa; el valor de MCP se aprecia cuando el modelo debe elegir entre varias herramientas, usar el resultado de una para construir la llamada siguiente y decidir cuándo detenerse. El escenario de la mesa de servicio se diseñó expresamente con esa forma.

La descripción de una herramienta forma parte de su interfaz. El modelo elige qué llamar leyendo el nombre, la descripción y el esquema de entrada, de modo que una descripción vaga produce llamadas incorrectas con la misma facilidad que un error de programación; redactarlas con cuidado es trabajo de ingeniería y no de documentación.

Esta primera parte deja el protocolo implementado y verificado en las dos direcciones que importan: un cliente propio que se entiende con servidores desarrollados por terceros, y un servidor propio que se entiende con ese mismo cliente. Sobre esa base, la segunda parte consiste en cambiar el transporte y observar el resultado en la red, sin modificar la lógica de las herramientas ni el anfitrión.